

BỘ GIÁO DỤC & ĐÀO TẠO
ĐẠI HỌC KINH TẾ THÀNH PHỐ HỒ CHÍ MINH
KHOA KINH TẾ



BÁO CÁO THỰC HÀNH LAB 04
ĐỀ TÀI: TẬP PHỎ BIẾN & LUẬT KẾT HỢP

MÔN HỌC: PHÂN TÍCH VÀ QUẢN LÝ ĐẦU TƯ NÂNG CAO

Giảng viên hướng dẫn: TS. Đỗ Như Tài
LHP: 26D2ECO50118301

Thành viên nhóm:

Võ Duy Huy Hoàng
Nguyễn Hữu Khoa
Hoàng Thị Hường

THÔNG TIN THÀNH VIÊN

Họ và tên	Mã số sinh viên	Tỷ trọng đóng góp
Võ Duy Huy Hoàng	87243020093	100%
Hoàng Thị Hường	88242020100	100%
Nguyễn Hữu Khoa	88234020092	100%

THÔNG TIN THÀNH VIÊN	2
A. MỤC TIÊU	4
B. DỮ LIỆU SỬ DỤNG	4
4.1. XÁC ĐỊNH TẬP PHỔ BIẾN VỚI GIẢI THUẬT APRIORI	4
4.1.1. Ôn tập lý thuyết	4
4.1.2. Bài làm mẫu	5
Bước 1–2: Cài thư viện và tính support/confidence/lift/leverage/conviction thủ công	5
Bước 3: Nạp, làm sạch dữ liệu Online Retail và biến đổi mỗi hóa đơn thành 1 dòng	5
Bước 4: Mã hóa dữ liệu về dạng one-hot cho giải thuật Apriori	6
Bước 5: Tìm tập phổ biến bằng giải thuật Apriori (3 case)	6
4.1.3. Bài tập trên lớp	7
Bước 1: Import thư viện, nạp và tiền xử lý dữ liệu	7
Bước 2: Tạo hàm để tìm tập phổ biến	7
Bước 3: Tìm tập phổ biến (tự cài đặt Apriori, tăng dần độ dài itemset)	7
4.1.4. Bài tập về nhà	8
4.2. XÁC ĐỊNH LUẬT KẾT HỢP TỪ TẬP PHỔ BIẾN	9
4.2.1. Ôn tập lý thuyết	9
4.2.2. Bài làm mẫu	9
Bước 1–2: Trích luật từ tập phổ biến (dùng case 3 ở 4.1.2)	9
Bước 3: Biểu đồ mối quan hệ giữa support và confidence	10
Bước 4: Biểu đồ phân bố tần suất confidence	10
4.2.3. Bài tập trên lớp	11
Bước 2: Xác định danh sách các tập luật ứng tuyển	11
Bước 3: Tính confidence từng luật ứng tuyển	11
Bước 4: Lọc các luật có confidence > 0.9	11
Bước 5–6: Top 5 luật có confidence cao nhất, ánh xạ tên phim	11
4.2.4. Bài tập về nhà	12
C. TÓM TẮT CHƯƠNG	14

A. MỤC TIÊU

- Triển khai tìm tập phổ biến bằng giải thuật Apriori (dùng thư viện mlxtend và tự cài đặt thuật toán).
- Trích xuất luật kết hợp từ tập phổ biến và trình bày luật kết hợp.
- Ôn tập lý thuyết về tập phổ biến và luật kết hợp.
- Thực hành với hai bộ dữ liệu thực tế: Online Retail (giao dịch bán lẻ) và MovieLens 100K (đánh giá phim).

B. DỮ LIỆU SỬ DỤNG

- Online Retail dataset (UCI Machine Learning Repository) — 541.909 giao dịch bán lẻ tại Anh, 01/12/2010 – 09/12/2011.
- MovieLens 100K dataset — 100.000 đánh giá của 943 người dùng trên 1682 phim.

4.1. XÁC ĐỊNH TẬP PHỔ BIẾN VỚI GIẢI THUẬT APRIORI

4.1.1. Ôn tập lý thuyết

Giải thuật Apriori hoạt động như thế nào?

Apriori là giải thuật kinh điển để tìm tập phổ biến (frequent itemsets), dựa trên tính chất Apriori property: nếu một tập hợp không phổ biến thì mọi tập cha (superset) của nó cũng không thể phổ biến. Các bước chính:

- Quét dữ liệu tìm các tập 1-itemset có support $\geq$ ngưỡng min_support.
- Từ các k-itemset phổ biến, sinh ứng viên (k+1)-itemset bằng cách kết hợp (join) các itemset có k-1 phần tử chung.
- Loại bỏ (prune) ứng viên có tập con k-itemset không phổ biến.
- Quét lại dữ liệu để tính support cho các ứng viên còn lại, giữ những ứng viên đạt ngưỡng.
- Lặp lại đến khi không còn ứng viên phổ biến ở độ dài kế tiếp.

Ý nghĩa của support, confidence, lift

- Support(X): tần suất xuất hiện của X trong toàn bộ giao dịch — đo độ phổ biến.
- Confidence($X \rightarrow Y$) = $\text{Support}(X \cup Y) / \text{Support}(X)$: xác suất Y xuất hiện khi đã có X — đo độ tin cậy của luật.
- Lift($X \rightarrow Y$) = $\text{Confidence}(X \rightarrow Y) / \text{Support}(Y)$: mức độ X, Y liên kết thực sự so với ngẫu nhiên độc lập. Lift > 1 nghĩa là tương quan dương.

Trong Apriori, support dùng để lọc tập phổ biến (giai đoạn sinh ứng viên); confidence và lift dùng ở giai đoạn sau để lọc ra các luật kết hợp mạnh, có ý nghĩa.

Khác biệt Apriori vs FP-Growth

Apriori sinh ứng viên tường minh và quét dữ liệu nhiều lần — chi phí I/O cao với dữ liệu lớn. FP-Growth nén dữ liệu vào cấu trúc cây FP-tree chỉ với 2 lần quét, khai thác đệ quy trên cây mà không cần sinh ứng viên tường minh, nên thường nhanh hơn đáng kể trên dữ liệu lớn hoặc support thấp.

Tiền xử lý dữ liệu giao dịch

Dữ liệu bán lẻ thường ở dạng long format (mỗi dòng là một sản phẩm trong một hóa đơn). Cần lọc bỏ dòng lỗi (số lượng ≤ 0 , hóa đơn hủy mã bắt đầu 'C'), loại giá trị thiếu, gom nhóm theo hóa đơn thành danh sách sản phẩm, rồi mã hóa one-hot bằng TransactionEncoder của mlxtend.

4.1.2. Bài làm mẫu

Bài toán 1: Tìm tập phổ biến trong dữ liệu giao dịch mua hàng tại siêu thị (Online Retail).

Bước 1–2: Cài thư viện và tính support/confidence/lift/leverage/conviction thủ công

```
!pip install mlxtend apyori --quiet
import matplotlib.pyplot as plt
import mlxtend.frequent_patterns
import mlxtend.preprocessing
import numpy
import pandas
```

```
example = [
    ['milk', 'bread', 'apples', 'cereal', 'jelly', 'cookies', 'salad', 'tomatoes'],
    ['beer', 'milk', 'chips', 'salsa', 'grapes', 'wine', 'potatoes', 'eggs', 'carrots'],
    ['diapers', 'baby formula', 'milk', 'bread', 'chicken', 'asparagus', 'cookies'],
    ['milk', 'cookies', 'chicken', 'asparagus', 'broccoli', 'cereal', 'orange juice'],
    ['steak', 'asparagus', 'broccoli', 'chips', 'salsa', 'ketchup', 'potatoes', 'salad'],
    ['beer', 'salsa', 'asparagus', 'wine', 'cheese', 'crackers', 'strawberries', 'cookies'],
    ['chocolate cake', 'strawberries', 'wine', 'cheese', 'beer', 'milk', 'orange juice'],
    ['chicken', 'peas', 'broccoli', 'milk', 'bread', 'eggs', 'potatoes', 'ketchup', 'crackers'],
    ['eggs', 'bread', 'cheese', 'turkey', 'salad', 'tomatoes', 'wine', 'steak', 'carrots'],
    ['bread', 'milk', 'tomatoes', 'cereal', 'chicken', 'turkey', 'chips', 'salsa', 'diapers']
]

N = len(example)
f_x = sum(['milk' in i for i in example])
f_y = sum(['bread' in i for i in example])
f_x_y = sum([all(w in i for w in ['milk', 'bread']) for i in example])

support = f_x_y / N
confidence = support / (f_x / N)
lift = confidence / (f_y / N)
leverage = support - ((f_x / N) * (f_y / N))
conviction = (1 - (f_y / N)) / (1 - confidence)

print("Support =", round(support, 4))
print("Confidence =", round(confidence, 4))
print("Lift =", round(lift, 4))
print("Leverage =", round(leverage, 4))
print("Conviction =", round(conviction, 4))
```

Kết quả thực hiện:

```
Support = 0.4
Confidence = 0.5714
Lift = 1.1429
Leverage = 0.05
Conviction = 1.1667
```

Bước 3: Nạp, làm sạch dữ liệu Online Retail và biến đổi mỗi hóa đơn thành 1 dòng

```
online = pandas.read_excel(io="Online Retail.xlsx", sheet_name="Online Retail", header=0)

online['IsCPresent'] = (
    online['InvoiceNo'].astype(str).apply(lambda x: 1 if x.find('C') != -1 else 0)
)

online1 = (
    online
    .loc[online['Quantity'] > 0]
    .loc[online['IsCPresent'] != 1]
    .loc[:, ['InvoiceNo', 'Description']]
    .dropna()
)
```

```

)

invoice_no_list = list(set(online1.InvoiceNo.tolist()))
print("Length of list of invoice numbers:", len(invoice_no_list))

subset_invoice_no_list = invoice_no_list[0:5000]
online1 = online1.loc[online1["InvoiceNo"].isin(subset_invoice_no_list)]

invoice_item_list = []
for num in list(set(online1.InvoiceNo.tolist())):
    tmp_df = online1.loc[online1["InvoiceNo"] == num]
    invoice_item_list.append(tmp_df.Description.tolist())

```

Kết quả thực hiện:

```
Length of list of invoice numbers: 20136
```

(Số hóa đơn hợp lệ thực tế trong Online Retail; giới hạn subset 5000 hóa đơn đầu để chạy nhanh hơn.)

Bước 4: Mã hóa dữ liệu về dạng one-hot cho giải thuật Apriori

```

online_encoder = mlxtend.preprocessing.TransactionEncoder()
online_encoder_array = online_encoder.fit_transform(invoice_item_list)
online_encoder_df = pandas.DataFrame(online_encoder_array, columns=online_encoder.columns_)
print("Data dimension:", online_encoder_df.shape)

```

Kết quả thực hiện:

```
Data dimension (row count, col count): (5000, 3703)
```

Bước 5: Tìm tập phổ biến bằng giải thuật Apriori (3 case)

```

# case 1: default min_support = 0.5, không dùng colnames
mod = mlxtend.frequent_patterns.apriori(online_encoder_df, low_memory=True)

# case 2: min_support = 0.01
mod_minsupport = mlxtend.frequent_patterns.apriori(
    online_encoder_df, min_support=0.01, low_memory=True)

# case 3: thêm tên cột để dễ đọc hơn
mod_colnames_minsupport = mlxtend.frequent_patterns.apriori(
    online_encoder_df, min_support=0.01, use_colnames=True, low_memory=True)
mod_colnames_minsupport['length'] = mod_colnames_minsupport['itemsets'].apply(len)

```

Kết quả thực hiện:

Case 1 (min_support mặc định = 0.5): DataFrame rỗng — không có sản phẩm nào được mua trong $\geq 50\%$ số hóa đơn (hợp lý vì dữ liệu bán lẻ có hàng nghìn sản phẩm đa dạng).

Case 2 & 3 (min_support = 0.01): tìm được 1876 tập phổ biến. Ví dụ 7 dòng đầu:

support	itemsets	length
0.0144	SET 2 TEA TOWELS I LOVE LONDON	1
0.0184	10 COLOUR SPACEBOY PEN	1
0.0132	12 MESSAGE CARDS WITH ENVELOPES	1
0.0170	12 PENCIL SMALL TUBE WOODLAND	1
0.0182	12 PENCILS SMALL TUBE RED RETROSPOT	1
0.0154	12 PENCILS SMALL TUBE SKULL	1

0.0132	12 PENCILS TALL TUBE RED RETROSPOT	1
--------	---------------------------------------	---

Lưu ý: tham số `low_memory=True` được thêm vào so với bản gốc để tránh tràn bộ nhớ khi tính toán trên 3703 cột sản phẩm.

4.1.3. Bài tập trên lớp

Bài toán 2: Tìm tập phổ biến cho bài toán đề xuất phim (MovieLens 100K)

Nhiệm vụ 1: Tìm tập phổ biến có trong tập dữ liệu phim

Bước 1: Import thư viện, nạp và tiền xử lý dữ liệu

```
import os
import pandas as pd

data_folder = "data/ml-100k"
ratings_filename = os.path.join(data_folder, "u.data")
all_ratings = pd.read_csv(ratings_filename, delimiter="\t", header=None,
                        names=["UserID", "MovieID", "Rating", "Datetime"])
all_ratings["Datetime"] = pd.to_datetime(all_ratings["Datetime"], unit='s')

all_ratings["Favorable"] = all_ratings["Rating"] > 3
ratings = all_ratings[all_ratings["UserID"].isin(range(200))]
favorable_ratings = ratings[ratings["Favorable"]]
favorable_reviews_by_users = dict((k, frozenset(v.values)) for k, v in
                                   favorable_ratings.groupby("UserID")["MovieID"])
num_favorable_by_movie = ratings[["MovieID", "Favorable"]].groupby("MovieID").sum()
num_favorable_by_movie.sort_values(by="Favorable", ascending=False).head()
```

Kết quả thực hiện:

MovieID	Favorable
50	100
100	89
258	83
181	79
174	74

Bước 2: Tạo hàm để tìm tập phổ biến

```
from collections import defaultdict

def find_frequent_itemsets(favorable_reviews_by_users, k_1_itemsets, min_support):
    counts = defaultdict(int)
    for user, reviews in favorable_reviews_by_users.items():
        for itemset in k_1_itemsets:
            if itemset.issubset(reviews):
                for other_reviewed_movie in reviews - itemset:
                    current_superset = itemset | frozenset((other_reviewed_movie,))
                    counts[current_superset] += 1
    return dict([(itemset, frequency) for itemset, frequency in
                counts.items() if frequency >= min_support])
```

Bước 3: Tìm tập phổ biến (tự cài đặt Apriori, tăng dần độ dài itemset)

```
import sys

frequent_itemsets = {}
min_support = 50

frequent_itemsets[1] = dict((frozenset((movie_id,)), row["Favorable"])
```

```

        for movie_id, row in num_favorable_by_movie.iterrows()
        if row["Favorable"] > min_support)
print("There are", len(frequent_itemsets[1]), "movies with more than", min_support, "favorable reviews")

for k in range(2, 20):
    cur = find_frequent_itemsets(favorable_reviews_by_users, frequent_itemsets[k-1], min_support)
    if len(cur) == 0:
        print("Did not find any frequent itemsets of length", k)
        break
    else:
        print("I found", len(cur), "frequent itemsets of length", k)
        frequent_itemsets[k] = cur

del frequent_itemsets[1]

```

Kết quả thực hiện:

```

There are 16 movies with more than 50 favorable reviews
I found 93 frequent itemsets of length 2
I found 295 frequent itemsets of length 3
I found 593 frequent itemsets of length 4
I found 785 frequent itemsets of length 5
I found 677 frequent itemsets of length 6
I found 373 frequent itemsets of length 7
I found 126 frequent itemsets of length 8
I found 24 frequent itemsets of length 9
I found 2 frequent itemsets of length 10
Did not find any frequent itemsets of length 11

```

Kết quả khớp hoàn toàn với tài liệu hướng dẫn.

4.1.4. Bài tập về nhà

Tìm hiểu thư viện `apyori` để tìm tập phổ biến từ dữ liệu mua hàng tại siêu thị — kết quả thực hành trình bày gộp cùng phần 4.2.4 bên dưới.

4.2. XÁC ĐỊNH LUẬT KẾT HỢP TỪ TẬP PHỔ BIẾN

4.2.1. Ôn tập lý thuyết

Luật kết hợp là gì?

Một luật kết hợp có dạng $X \rightarrow Y$ (X, Y là các tập sản phẩm rời nhau): 'nếu giao dịch chứa X thì có khả năng cũng chứa Y '. Ba chỉ số đánh giá luật:

- $\text{Support}(X \rightarrow Y) = \text{Support}(X \cup Y)$: tỷ lệ giao dịch chứa cả X và Y .
- $\text{Confidence}(X \rightarrow Y) = \text{Support}(X \cup Y) / \text{Support}(X)$: xác suất có Y khi đã có X .
- $\text{Lift}(X \rightarrow Y) = \text{Confidence}(X \rightarrow Y) / \text{Support}(Y)$: mức độ liên kết thực sự so với ngẫu nhiên.

Quy trình từ tập phổ biến sang luật kết hợp

- Với mỗi tập phổ biến F ($|F| \geq 2$), sinh mọi cách chia $F = \text{premise} \cup \text{conclusion}$ (rời nhau).
- Tính $\text{confidence} = \text{Support}(F) / \text{Support}(\text{premise})$ cho mỗi cách chia.
- Giữ lại các luật vượt ngưỡng min_threshold (theo confidence hoặc lift) đã chọn.

Luật 'mạnh' (strong rule)

Một luật được coi là mạnh khi vừa có confidence cao vừa có $\text{lift} > 1$ (trương quan thực sự, không phải do một sản phẩm vốn đã rất phổ biến). Confidence cao nhưng $\text{lift} \sim 1$ có thể không mang nhiều giá trị gợi ý thực sự.

4.2.2. Bài làm mẫu

Bài toán 1: Xác định luật kết hợp trong dữ liệu giao dịch Online Retail

Bước 1–2: Trích luật từ tập phổ biến (dùng case 3 ở 4.1.2)

```
# case 1: hiển thị các luật có confidence >= 0.6
rules = mlxtend.frequent_patterns.association_rules(
    mod_colnames_minsupport, metric="confidence", min_threshold=0.6, support_only=False)

# case 2: liệt kê các luật thỏa điều kiện lift >= 50
rules2 = mlxtend.frequent_patterns.association_rules(
    mod_colnames_minsupport, metric="lift", min_threshold=50, support_only=False)
print("Number of Associations:", rules2.shape[0])
```

Kết quả case 1 ($\text{confidence} \geq 0.6$): 549 luật kết hợp. Ví dụ:

antecedents	consequents	support	confidence	lift
ALARM CLOCK BAKELIKE CHOCOLATE	ALARM CLOCK BAKELIKE GREEN	0.0174	0.725	13.58
ALARM CLOCK BAKELIKE CHOCOLATE	ALARM CLOCK BAKELIKE RED	0.0184	0.767	13.17
ALARM CLOCK BAKELIKE IVORY	ALARM CLOCK BAKELIKE GREEN	0.0188	0.631	11.81
ALARM CLOCK	ALARM CLOCK	0.0152	0.724	13.55

BAKELIKE ORANGE	BAKELIKE GREEN			
ALARM CLOCK BAKELIKE RED	ALARM CLOCK BAKELIKE GREEN	0.0380	0.653	12.23

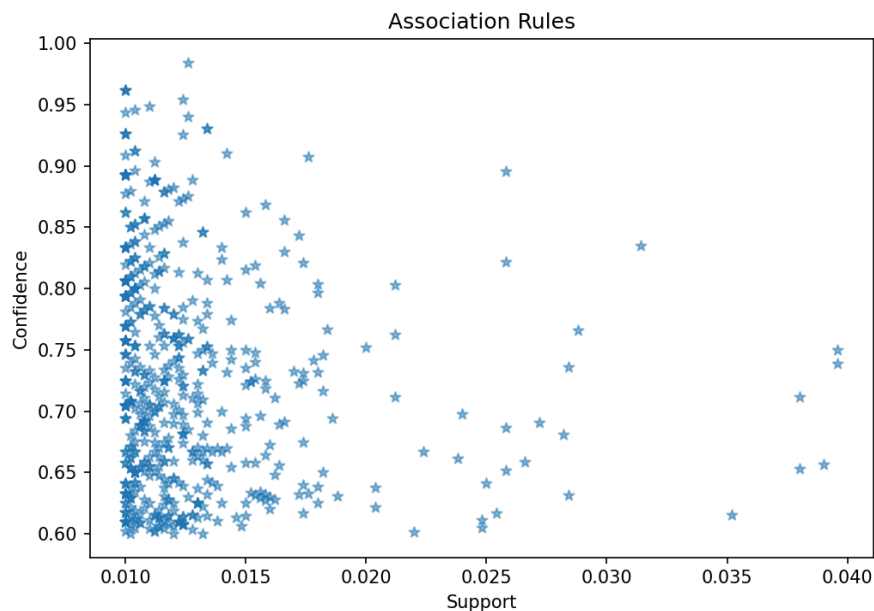
→ Khách mua đồng hồ báo thức Bakelike màu này thường mua kèm màu khác cùng dòng sản phẩm (bộ sưu tập).

Kết quả case 2 (lift ≥ 50): Number of Associations = 48.

Nổi bật là các bộ 'HERB MARKER' (biển đánh dấu cây gia vị: basil, mint, parsley, rosemary, thyme...) với lift 68–70, cho thấy các sản phẩm này gần như luôn được mua theo bộ.

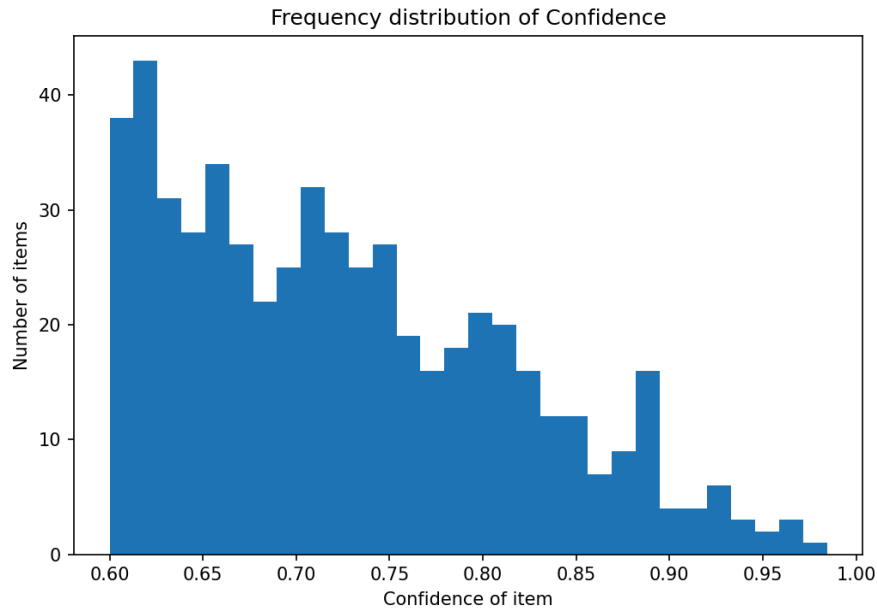
Bước 3: Biểu đồ mối quan hệ giữa support và confidence

```
rules.plot.scatter("support", "confidence", alpha=0.5, marker="*")
plt.xlabel("Support"); plt.ylabel("Confidence"); plt.title("Association Rules")
plt.show()
```



Bước 4: Biểu đồ phân bố tần suất confidence

```
rules.hist("confidence", grid=False, bins=30)
plt.xlabel("Confidence of item"); plt.ylabel("Number of items")
plt.title("Frequency distribution of Confidence")
plt.show()
```



4.2.3. Bài tập trên lớp

Bài toán 2: Tìm luật kết hợp trong tập dữ liệu phim (MovieLens 100K)

Bước 2: Xác định danh sách các tập luật ứng tuyển

```
candidate_rules = []
for itemset_length, itemset_counts in frequent_itemsets.items():
    for itemset in itemset_counts.keys():
        for conclusion in itemset:
            premise = itemset - set((conclusion,))
            candidate_rules.append((premise, conclusion))
print("There are", len(candidate_rules), "candidate rules")
```

Kết quả thực hiện:

There are 15285 candidate rules

Bước 3: Tính confidence từng luật ứng tuyển

```
correct_counts = defaultdict(int)
incorrect_counts = defaultdict(int)
for user, reviews in favorable_reviews_by_users.items():
    for candidate_rule in candidate_rules:
        premise, conclusion = candidate_rule
        if premise.issubset(reviews):
            if conclusion in reviews:
                correct_counts[candidate_rule] += 1
            else:
                incorrect_counts[candidate_rule] += 1

rule_confidence = {r: correct_counts[r] / float(correct_counts[r] + incorrect_counts[r])
                  for r in candidate_rules}
```

Bước 4: Lọc các luật có confidence > 0.9

```
min_confidence = 0.9
rule_confidence = {r: c for r, c in rule_confidence.items() if c > min_confidence}
print(len(rule_confidence))
```

Kết quả thực hiện:

5152

Bước 5–6: Top 5 luật có confidence cao nhất, ánh xạ tên phim

```
from operator import itemgetter
```

```

sorted_confidence = sorted(rule_confidence.items(), key=itemgetter(1), reverse=True)

movie_name_data = pd.read_csv(os.path.join(data_folder, "u.item"), delimiter="|",
                               header=None, encoding="mac-roman")
movie_name_data.columns = ["MovieID", "Title", "Release Date", "Video Release", "IMDB", "<UNK>",
                           "Action", "Adventure", "Animation", "Children's", "Comedy", "Crime", "Documentary", "Drama",
                           "Fantasy", "Film-Noir", "Horror", "Musical", "Mystery", "Romance", "Sci-Fi", "Thriller", "War", "Western"]

def get_movie_name(movie_id):
    return movie_name_data[movie_name_data["MovieID"] == movie_id]["Title"].values[0]

for index in range(5):
    premise, conclusion = sorted_confidence[index][0]
    premise_names = ", ".join(get_movie_name(idx) for idx in premise)
    conclusion_name = get_movie_name(conclusion)
    print("Rule #{}: If a person recommends {} they will also recommend {}".format(
        index+1, premise_names, conclusion_name))
    print(" - Confidence: {:.3f}".format(rule_confidence[(premise, conclusion)]))

```

Kết quả thực hiện:

#	Premise (phim đã thích)	→ Conclusion	Confidence
1	Silence of the Lambs, The (1991); Return of the Jedi (1983)	Star Wars (1977)	1.000
2	Empire Strikes Back, The (1980); Fugitive, The (1993)	Raiders of the Lost Ark (1981)	1.000
3	Contact (1997); Empire Strikes Back, The (1980)	Raiders of the Lost Ark (1981)	1.000
4	Toy Story (1995); Return of the Jedi (1983); Twelve Monkeys (1995)	Star Wars (1977)	1.000
5	Toy Story (1995); Empire Strikes Back, The (1980); Twelve Monkeys (1995)	Raiders of the Lost Ark (1981)	1.000

Kết quả hợp lý về mặt nội dung: người thích các phim khoa học viễn tưởng kinh điển (Star Wars, Empire Strikes Back, Raiders of the Lost Ark, Return of the Jedi) có xu hướng thích cả bộ — phù hợp trực giác vì đây đều là các phim cùng thời, cùng thể loại, được yêu thích bởi nhóm khán giả tương tự.

4.2.4. Bài tập về nhà

Tìm hiểu thư viện `apyori` để tìm luật kết hợp từ dữ liệu mua hàng tại siêu thị

Khác với `mlxtend` (tách riêng bước `apriori()` tìm tập phổ biến và bước `association_rules()` trích luật), `apyori` gộp cả hai bước vào một lệnh gọi `apriori()` duy nhất, nhận trực tiếp danh sách giao dịch và trả về các luật thỏa các ngưỡng `min_support`, `min_confidence`, `min_lift`.

```

from apyori import apriori as apyori_apriori

results = list(apyori_apriori(
    invoice_item_list,
    min_support=0.02, min_confidence=0.4, min_lift=3, min_length=2
))
print("Số luật tìm được:", len(results))
for r in results[:5]:
    print(list(r.items), "support=", round(r.support, 4))
    for stat in r.ordered_statistics:
        print(" ", set(stat.items_base), "->", set(stat.items_add),
              "conf=", round(stat.confidence, 3), "lift=", round(stat.lift, 3))

```

Kết quả thực hiện (chạy trên 2000 hóa đơn để đảm bảo tốc độ):

```

Số hóa đơn dùng cho apyori: 2000
Số luật tìm được: 90

['60 TEATIME FAIRY CAKE CASES', 'PACK OF 60 PINK PAISLEY CAKE CASES'] support= 0.0225
{60 TEATIME FAIRY CAKE CASES} -> {PACK OF 60 PINK PAISLEY CAKE CASES} conf= 0.495
lift= 9.696
{PACK OF 60 PINK PAISLEY CAKE CASES} -> {60 TEATIME FAIRY CAKE CASES} conf= 0.441
lift= 9.696

['ALARM CLOCK BAKELIKE GREEN', 'ALARM CLOCK BAKELIKE RED '] support= 0.0375
{ALARM CLOCK BAKELIKE GREEN} -> {ALARM CLOCK BAKELIKE RED } conf= 0.658 lift=
10.697
{ALARM CLOCK BAKELIKE RED } -> {ALARM CLOCK BAKELIKE GREEN} conf= 0.61 lift=
10.697

```

Các luật tìm được bằng apyori nhất quán với các luật tìm được bằng mlxtend ở phần 4.2.2 (cùng phát hiện nhóm 'ALARM CLOCK BAKELIKE' các màu thường được mua kèm nhau), qua đó xác nhận tính đúng đắn của cả hai cách tiếp cận.

C. TÓM TẮT CHƯƠNG

- Tìm tập phổ biến với Apriori: dùng mlxtend trên Online Retail (5000 hóa đơn, tìm được 1876 tập phổ biến với $\text{min_support}=0.01$), và tự cài đặt Apriori bằng Python thuần trên MovieLens 100K (hơn 2900 tập phổ biến qua các độ dài từ 2 đến 10).
- Trích luật kết hợp: 549 luật ($\text{confidence} \geq 0.6$) và 48 luật ($\text{lift} \geq 50$) trên Online Retail, phản ánh đúng hành vi mua theo bộ sưu tập/combo sản phẩm. Trên MovieLens: 5152 luật $\text{confidence} > 0.9$, top 5 luật cho thấy nhóm phim khoa học viễn tưởng kinh điển thường được yêu thích cùng nhau.
- Bài tập về nhà: minh họa thư viện apyori — gộp bước tìm tập phổ biến và trích luật vào một lệnh gọi, cho kết quả nhất quán với mlxtend.

Việc thực hành trên cả hai bộ dữ liệu (giao dịch bán lẻ và đánh giá phim) giúp hiểu rõ Apriori là kỹ thuật tổng quát, áp dụng được cho bất kỳ bài toán nào có dạng 'tập các item được chọn cùng nhau', không chỉ giới hạn ở market basket analysis truyền thống.